Нагрузочный тест

Цель: подтвердить, что стенд на одном сервере (организаторы: ~8 CPU, 32–64 ГБ, без GPU) выдерживает десятки одновременных пользователей с приемлемым временем основных операций (жёсткого SLA нет), и найти узкие места.

Метод

- Инструмент: Locust 2.46 (MIT), сценарий `loadtest/locustfile.py`. Запуск против контейнерного стенда через `web` (nginx → backend), то есть через тот же путь, что у браузера.
- Смесь виртуальных пользователей (веса 5 : 2 : 3):
  - **кандидат**: вход → профиль → опрос (4 шага) → запуск теста → ответы на все задания адаптивной сессии с паузами «читает задание» → результат → квалификация; после теста — просмотр (профиль, приглашения, вакансии);
  - **работодатель**: вход → разбор брифа (`parse`) → создание брифа → категории и охват вилки → проверка навыков → сборка подборки → подборка → карточка кандидата → шорт-лист → приглашение; банк кандидатов и сводка, список брифов и воронка приглашений;
  - **гость**: главная и её ассеты, SPA-маршруты, `/api/health` , `/api/fsp/availability` , `/api/docs` .
- Паузы между действиями 1–3 с (кандидат), 2–6 с (работодатель), 1–5 с (гость). У каждого виртуального пользователя свой `X-Forwarded-For` (лимиты входа считаются по IP, а все запросы идут с одной машины): для этого на стенде `VZ_REAL_IP_FROM=172.16.0.0/12` .
- Сбой = 5xx, таймаут или неожиданный 4xx. 409/422/429 на приглашение (лимиты и «активная пара») и 409/422 на ответ в тесте считаются штатными.
- Аккаунты: `backend/scripts/loadtest_accounts.py` создаёт подтверждённых кандидатов `lt0001@loadtest.example` и HR-аккаунты `lthr001@…` при демо-компаниях (лимит запросов подбора считается на аккаунт, поэтому у виртуальных HR свои логины). Для каждого прогона берётся свежая партия кандидатов (`--offset`), потому что повторный тест на том же аккаунте ограничен.
- Длительность прогона 3 минуты, разгон 5–10 пользователей в секунду.

Как повторить

```
docker compose up -d --build && make demo                  # стенд и демо-мир (см. DEPLOY.md)
# для нагрузки с хоста nginx должен верить X-Forwarded-For от хоста (только для теста):
VZ_REAL_IP_FROM=172.16.0.0/12 docker compose up -d web
make loadtest-accounts                                     # аккаунты → loadtest/accounts.json (в
.gitignore)
make loadtest USERS=50 RUNTIME=3m                          # отчёт: loadtest/out/u50.html и u50_stats.csv
docker compose exec -T backend python scripts/loadtest_accounts.py --candidates 300 --offset 1000 --out
var/loadtest_accounts.json   # свежая партия
```

`make loadtest` запускает Locust через `uvx` (нужен uv); без него — `pip install -r loadtest/requirements.txt` и `locust -f loadtest/locustfile.py --headless -u 50 -r 5 -t 3m --host http://localhost` .

Окружение замеров

Windows 11, Docker Desktop (WSL2), 20 логических ядер, 11 ГБ памяти для Docker; Locust на том же хосте (он тоже потребляет CPU, поэтому цифры слегка занижены относительно отдельной нагрузочной машины). Отдельный проект compose, чистая БД с демо-миром (300 синтетических кандидатов, пул 18 200 экземпляров).

Конфигурация стенда: 4 процесса uvicorn (`VZ_WORKERS=4`), пул БД 8+4 на процесс, PostgreSQL `max_connections=200` , `shared_buffers=256MB` .

Результаты (итоговая конфигурация, 4 процесса)

Пользователей	Запросов	Сбоев	RPS (в среднем)	p50	p95	p99
50	13 075	0	≈ 73	26 мс	180 мс	400 мс
100	22 263	0	≈ 125	74 мс	480 мс	1,1 с

p50 / p95 / p99, мс, по основным операциям:

Операция	50: p50	50: p95	50: p99	100: p50	100: p95	100: p99
`GET /` (index.html)	3	9	13	3	14	34
`POST /api/auth/login` (Argon2)	180	320	430	390	930	1100
`GET /api/auth/me`	14	42	67	46	170	360
`PUT /api/survey/steps/stack`	66	120	230	78	220	260
`POST /api/tests/sessions` (старт теста)	78	150	240	110	330	590
`GET /api/tests/sessions/{id}/next`	70	290	370	140	430	870
`POST /api/tests/sessions/{id}/answer`	48	140	270	86	320	510
`GET /api/tests/sessions/{id}/result`	37	130	140	63	170	260
`GET /api/invitations/inbox` (кандидат)	18	50	88	60	220	410
`POST /api/needs/parse`	26	82	100	72	240	440
`POST /api/needs/{id}/versions` (сборка подборки)	220	440	510	470	1100	1400
`GET /api/needs/{id}/selection`	63	220	330	210	700	1100
`POST /api/invitations`	140	380	640	430	1200	1500
`GET /api/needs`	53	260	320	110	380	470
`GET /api/bank`	160	390	470	430	1100	1700
`GET /api/invitations` (воронка работодателя), **до исправления**	590	1400	1700	1900	4900	6500
`GET /api/invitations` (воронка работодателя), **после исправления**	не измерялось	—	—	65	150	200

Ресурсы (`docker stats` , 4-секундная выборка за прогон):

Сервис	50 польз.: CPU ср. / макс.	50: память макс.	100 польз.: CPU ср. / макс.	100: память макс.
backend (4 процесса)	141 % / 354 %	1,0 ГБ	353 % / 859 %	1,4 ГБ
db	38 % / 77 %	305 МБ	78 % / 137 %	378 МБ
web (nginx)	6 % / 14 %	17 МБ	14 % / 49 %	20 МБ
fsp-mock	3 % / 36 %	47 МБ	8 % / 84 %	37 МБ

100 % = одно ядро. На 8-ядерном сервере 100 пользователей занимают около 4,5 ядер из 8.

Что нашли и исправили

Прогон	Наблюдение	Причина	Исправление
1 процесс, 50 польз.	177 ответов 429 на вход	все запросы шли с одним IP: nginx отдавал бэкенду только адрес прокси	`X-Forwarded-For` теперь идёт от настоящего клиента (`real_ip` в nginx, доверие по `VZ_REAL_IP_FROM`); в тесте у каждого пользователя свой адрес
1 процесс, 50 польз.	8 ответов 500 на `GET/POST /api/invitations`	**deadlock** в PostgreSQL: `refresh_rows` блокировал ожидающие приглашения `FOR UPDATE` в произвольном порядке, а чтения списков идут параллельно (и с фоновым sweep)	строки блокируются в порядке id; блокировка берётся только если срок/напоминание действительно наступили (пробный прогон без блокировки) — `app/services/contact/store.py`
1 процесс, 50 польз.	429 `rate_limited` на `/api/needs/*`	14 виртуальных HR на 6 аккаунтах, лимит 30 запросов/мин на аккаунт	не дефект: у тестовых HR свои аккаунты (40)
1 процесс, 100 польз.	p50 ≈ 1 с даже у `/api/health` , 10 тыс. запросов за 3 мин	один процесс Python упирается в одно ядро (GIL), CPU 120–290 % не помогает	`VZ_WORKERS=4` (uvicorn), пул БД 8+4 на процесс, `max_connections=200` ; часы демо синхронизируются между процессами через БД
4 процесса	`GET /api/needs` 440 мс на 90 брифов	N+1: два запроса на каждый бриф (183 запроса)	один запрос брифов + один запрос версий + один группировки закреплённых — `app/services/matching/needs.py` , тест `tests/test_list_needs_batched.py`

Сводка по этапам (100 пользователей): 1 процесс — p50 1000 мс, p95 2100 мс, p99 3200 мс, 10 тыс. запросов; 4 процесса — p50 74 мс, p95 480 мс, p99 1100 мс, 22 тыс. запросов (рост пропускной способности вдвое при вдвое меньших паузах ожидания клиентов).

Воронка приглашений: пакетная загрузка и постраничность

Что	Было	Стало
Запросов на `GET /api/invitations`	≈ 5 на приглашение (условия, события, раскрытие, проверка исхода, псевдоним) + чтение всего списка	константа на страницу: 19 запросов при 2 и при 6 приглашениях (тест), из них ≈ 12 — общие (аутентификация, квота, компания)
Размер ответа	все приглашения компании	страница `limit` (по умолчанию 50, максимум 200) с курсором `next_cursor` по ключу `(created_at, id)`
Счётчики колонок, воронка, причины отказов, средняя позиция, мини-воронки по брифам	считались по загруженным строкам	агрегатом SQL по **всем** приглашениям (один запрос), не зависят от страницы
Входящие кандидата (`/inbox`), отклики (`/applications/mine` , список работодателя)	≈ 3–5 запросов на строку, компания и R читались на каждую строку	условия, события, раскрытия, вакансии, псевдонимы, компании и события надёжности грузятся пакетом; `/inbox` принимает необязательные `limit` и `cursor` , `new_count` — агрегат
Применение сроков при чтении	пробный `to_domain` на каждую ожидающую строку	то же, но по пакету из 4 запросов на список

Где: `app/services/contact/store.py` (`prefetched` , `refresh_rows`), `invitations.py` (`employer_page` , `employer_cards` , `kanban` , `candidate_cards`), `applications.py` , `identity.aliases` , `reliability.events_by_company` . Тест: `tests/test_list_invitations_batched.py` (число запросов не растёт с длиной списка для воронки, входящих, откликов; курсор не теряет и не дублирует строки; счётчики по всем строкам). Фронтенд: `useKanban` — бесконечный запрос (кнопка «Показать ещё» на странице «Приглашения»), заголовки колонок берут `count` из агрегата.

Перепрогон после исправления (2026-10-08, 100 пользователей, 3 минуты, отдельный проект compose `vzlt` на портах 8100/8180/5543, свежие образы, демо-мир 315 кандидатов, пул ≈ 12 900 экземпляров, 4 процесса uvicorn; Locust на том же хосте): **24 739 запросов, 0 сбоев, ≈ 140 RPS, p50 31 мс, p95 170 мс, p99 300 мс** в целом. Воронка работодателя `GET /api/invitations` (197 запросов): **p50 65 мс, p95 150 мс, p99 200 мс** против 1900 / 4900 / 6500 мс до исправления (≈ 30-кратное снижение медианы). Остальное в том же диапазоне, что в таблице выше: `POST /api/auth/login` 160 / 260 / 630, `GET /api/invitations/inbox` 25 / 86 / 130, `GET /api/needs` 30 / 91 / 120, `GET /api/bank` 140 / 330 / 400, `POST /api/needs/{id}/versions` 220 / 400 / 460, `POST /api/invitations` 140 / 320 / 410 (p50 / p95 / p99, мс). Прогон при 50 пользователях после исправления не повторялся; замеры ресурсов (`docker stats`) в этом прогоне не снимались. Отчёт: `loadtest/out/u100.html` .

Выводы

- 50 одновременных пользователей: 0 сбоев, медиана 26 мс, p99 400 мс, включая воронку приглашений после исправления. 100 одновременных: 0 сбоев, медиана 74 мс, p95 0,5 с, p99 1,1 с. Запас по CPU на сервере класса 8 CPU примерно двукратный.
- Самые тяжёлые операции: сборка подборки (`versions` , 220–470 мс медиана — Score по всему пулу, его и ожидаем), вход (Argon2, 180–390 мс — сознательная цена защиты паролей), банк и воронка приглашений.
- **Было узким местом: `GET /api/invitations` (воронка работодателя)** — список не постраничный, ≈ 5 запросов на каждое приглашение; у компании с сотнями приглашений 100 пользователей давали p50 1,9 с, p99 6,5 с. **Исправлено и перемерено**: p50 65 мс, p99 200 мс при 100 пользователях (см. раздел «Воронка приглашений»).
- Лимиты запросов (`RateLimiter`) и буфер `/api/demo/webhook-sink` хранятся в памяти процесса: при `VZ_WORKERS=4` эффективные лимиты до 4 раз мягче. Часы демо (`/api/demo/clock`) синхронизируются через БД раз в секунду.
- Фоновый планировщик защищён рекомендательной блокировкой PostgreSQL, несколько процессов не дублируют задачи.
- Если контейнер упал, `restart: unless-stopped` поднимет его; при падении одного из 4 процессов uvicorn остальные продолжают работать.

Что не проверено

- Нагрузка с отдельной машины и по сети; сервер класса 8 CPU / 32+ ГБ (замеры на ноутбуке с 20 логическими ядрами и 11 ГБ).
- Более 100 одновременных пользователей и длительные (часы) прогоны; рост БД после тысяч брифов и приглашений.
- Боевая схема с Caddy и HTTPS (`docker-compose.prod.yml`): собран и проверен только конфиг; цепочка Caddy → nginx → backend для клиентского IP (`VZ_REAL_IP_FROM=172.29.80.0/24`) не прогонялась.
- Синхронизация демо-часов между процессами проверена только чтением кода и тестами модуля, живая проверка кнопкой «+1 день» при 4 процессах не выполнена.